

M71V MAESTRÍA EN GESTIÓN Y ANÁLISIS DE DATOS FINANCIEROS
MÓDULO M71V/M72V 02 – IMPLEMENTACIÓN DE MODELOS DE APRENDIZAJE
AUTOMÁTICO

TRABAJO FINAL – ENTREGA 3
DESPLIEGUE PARA CONSUMO

PREDICCIÓN DE RENUNCIAS DE EMPLEADOS
(IBM HR ANALYTICS EMPLOYEE ATTRITION)

Docente:

Barreto Marcano, Aquiles

Alumno:

Raffo Romero, Giovanni Antonio

SEPTIEMBRE 2026

RESUMEN EJECUTIVO

El presente informe corresponde a la tercera y última entrega del trabajo final: la puesta en funcionamiento del modelo entrenado en la Entrega 2, de modo que el área de Recursos Humanos pueda consultarlo con los datos de un empleado y recibir una respuesta inmediata.

El modelo se expone como un servicio web construido con FastAPI, con dos direcciones: una de control, que informa si el servicio está vivo y si cargó el modelo, y una de predicción, que recibe los 29 datos de un empleado y devuelve si se lo considera en riesgo de renuncia junto con la probabilidad estimada. El servicio no vuelve a programar la preparación de los datos: carga el mismo archivo guardado en la Entrega 2, que contiene la receta de preparación, el modelo y el punto de corte en un único objeto, por lo que la transformación en producción es idéntica a la del entrenamiento por construcción.

El contrato de entrada está validado campo por campo: las siete variables de texto solo admiten los valores presentes en el dataset y cualquier otro valor se rechaza con un error explícito antes de llegar al modelo. El servicio se probó con tres empleados reales del grupo de examen, elegidos para cubrir los tres casos que Recursos Humanos va a encontrar: riesgo alto, empleado estable y caso de frontera. La documentación interactiva se genera sola y sirve de demostración en el navegador. El cuaderno de esta entrega, que levanta el servicio y lo consulta sin necesidad de instalar nada, puede ejecutarse en Google Colab desde el enlace de la sección 7.

ÍNDICE

1. INTRODUCCIÓN	4
2. ARQUITECTURA DE LA SOLUCIÓN	4
2.1 Un único artefacto	4
2.2 Carga única al arrancar	4
2.3 Las dos direcciones del servicio	4
3. CONTRATO DE ENTRADA Y SALIDA	5
3.1 Entrada	5
3.2 Salida	6
4. PRUEBA CON TRES EMPLEADOS	6
5. VALIDACIÓN DE LAS ENTRADAS	7
6. EJECUCIÓN Y DEMOSTRACIÓN	7
7. CONCLUSIONES	8
8. LINK COLAB	8
9. REFERENCIAS	9

1. INTRODUCCIÓN

La Entrega 1 dejó los datos preparados y un grupo de examen apartado antes de cualquier transformación. La Entrega 2 entrenó un único modelo, una regresión logística con pesos de clase, lo ajustó por validación cruzada, fijó el punto de corte en 0,40 con criterio de negocio y guardó todo en un único archivo: `models/modelo_attrition.pkl`.

Un modelo guardado en un archivo no sirve a nadie por sí solo. Para que Recursos Humanos pueda usarlo hace falta un servicio que reciba los datos de un empleado y devuelva una respuesta. Esta entrega cubre ese paso: la construcción de una interfaz de programación (API) local con FastAPI, la definición del formato de entrada y de salida, su validación, las pruebas con ejemplos y la documentación.

El criterio que ordena toda la entrega es uno solo: que la preparación de los datos en producción sea exactamente la misma que en el entrenamiento. No se logra copiando código, sino reutilizando el mismo objeto.

2. ARQUITECTURA DE LA SOLUCIÓN

2.1 Un único artefacto

El archivo `models/modelo_attrition.pkl` contiene tres cosas dentro de un mismo objeto: la receta de preparación (estandarización de las variables numéricas y codificación de las categóricas), el modelo entrenado con su configuración y el punto de corte de 0,40. El servicio lo carga con una sola línea y le pasa los datos crudos del empleado, tal como están en la tabla de Recursos Humanos.

De ahí se desprende la garantía que exige la consigna: el servicio no reimplementa ningún paso del preprocesamiento, de modo que no existe la posibilidad de que la transformación en inferencia difiera de la del entrenamiento. Si mañana se reentrena el modelo, se reemplaza el archivo y el servicio no cambia.

2.2 Carga única al arrancar

El modelo se carga una sola vez, cuando el servicio arranca, y queda en memoria para todas las consultas siguientes (mecanismo lifespan de FastAPI). Leer el archivo en cada consulta funcionaría, pero multiplicaría el tiempo de respuesta sin ninguna ventaja.

2.3 Las dos direcciones del servicio

Tabla 1: Rutas expuestas por el servicio

Ruta	Tipo	Para qué sirve
<code>/health</code>	GET	Responder si el servicio está vivo y si el modelo quedó cargado. Es lo que consultaría un monitor automático.
<code>/predict</code>	POST	Recibir los datos de un empleado y devolver la decisión y la probabilidad de renuncia.
<code>/docs</code>	GET	Documentación interactiva, generada sola por FastAPI a partir del código. Permite probar el servicio desde el navegador.

Fuente: elaboración propia

No se agregaron rutas adicionales. La consigna pide un endpoint de predicción; el de control es el mínimo necesario para operar el servicio y el de documentación lo genera la propia herramienta.

Gráfico 1: Documentación interactiva del servicio (pantalla /docs)



Fuente: elaboración propia en Python

3. CONTRATO DE ENTRADA Y SALIDA

3.1 Entrada

La entrada es un empleado descrito con sus 29 variables crudas, en formato JSON. Son las mismas que espera el modelo: las 35 columnas originales menos las cuatro descartadas en la Entrega 1 (tres constantes y el número de legajo), menos el nivel de puesto, eliminado por ser casi idéntico al ingreso mensual, y menos la columna de renuncia, que es justamente lo que se predice.

Cada campo se declara con su tipo y las siete variables de texto se restringen a los valores que existen en el dataset. La Tabla 2 resume el contrato.

Tabla 2: Formato de entrada (29 campos)

Grupo	Campos	Valores admitidos
Datos personales y de contrato	Age, Gender, MaritalStatus, DistanceFromHome, Education, EducationField	Números enteros; Gender: Female o Male; MaritalStatus: Divorced, Married o Single
Puesto y área	Department, JobRole, BusinessTravel, OverTime	Solo los valores del dataset: 3 departamentos, 9 puestos, 3 modalidades de viaje, OverTime No o Yes
Remuneración	MonthlyIncome, DailyRate, HourlyRate, MonthlyRate, PercentSalaryHike, StockOptionLevel	Números enteros
Trayectoria	TotalWorkingYears, YearsAtCompany, YearsInCurrentRole, YearsSinceLastPromotion,	Números enteros

Grupo	Campos	Valores admitidos
	YearsWithCurrManager, NumCompaniesWorked, TrainingTimesLastYear	
Escalas de 1 a 5	EnvironmentSatisfaction, JobSatisfaction, JobInvolvement, RelationshipSatisfaction, WorkLifeBalance, PerformanceRating	Números enteros, tratados como números según la decisión de la Entrega 1

Fuente: elaboración propia

Gráfico 2: Formato de entrada en la documentación interactiva, con un empleado de ejemplo

POST /predict Predict

Parameters Try it out

No parameters

Request body required application/json

Example Value Schema

```
{
  "Age": 21,
  "BusinessTravel": "Travel_Frequently",
  "DailyRate": 756,
  "Department": "Sales",
  "DistanceFromHome": 1,
  "Education": 1,
  "EducationField": "Technical Degree",
  "EnvironmentSatisfaction": 1,
  "Gender": "Female",
  "HourlyRate": 90,
  "JobInvolvement": 2,
  "JobRole": "Sales Representative",
  "JobSatisfaction": 2,
  "MaritalStatus": "Single",
  "MonthlyIncome": 2174,
  "MonthlyRate": 9150,
  "NumCompaniesWorked": 1,
  "OverTime": "Yes",
  "PercentSalaryHike": 11,
  "PerformanceRating": 3,
  "RelationshipSatisfaction": 3,
  "StockOptionLevel": 0,
  "TotalWorkingYears": 1,
  "TrainingTimesLastYear": 3,
  "WorkLifeBalance": 3,
  "YearsAtCompany": 1,
  "YearsInCurrentRole": 0,
}
```

Fuente: elaboración propia en Python

3.2 Salida

La respuesta tiene dos campos: attrition, con el valor Yes o No, y probabilidad, la probabilidad estimada de renuncia entre 0 y 1. Por ejemplo: {"attrition": "Yes", "probabilidad": 0.9877}.

La decisión Yes o No no se calcula en el servicio comparando la probabilidad contra un número escrito a mano: la devuelve el propio modelo, que lleva el corte de 0,40 fijado adentro desde la Entrega 2. El servicio no conoce el corte ni necesita conocerlo. Se informan las dos cosas porque Recursos Humanos las usa distinto: la decisión sirve para armar la lista de empleados a conversar y la probabilidad para priorizar por dónde empezar.

4. PRUEBA CON TRES EMPLEADOS

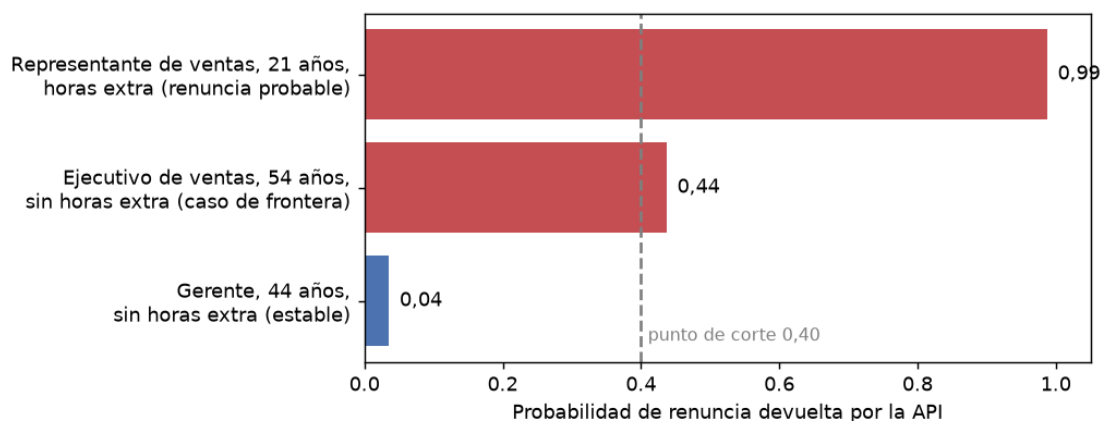
El servicio se probó con tres empleados reales del grupo de examen, que el modelo nunca vio durante el entrenamiento, elegidos para cubrir los tres casos que Recursos Humanos va a encontrar en la práctica. Sus datos están en el archivo api/ejemplos.json y el script api/probar_ejemplos.py los envía al servicio y muestra las respuestas.

Tabla 3: Respuestas del servicio a los tres empleados de prueba

Caso	Perfil	Probabilidad	Respuesta	Qué pasó realmente
Riesgo alto	Representante de ventas, 21 años, horas extra, viaja seguido, ingreso 2.174	0,99	Yes	Renunció
Estable	Gerente, 44 años, sin horas extra, ingreso 19.190	0,04	No	Se quedó
Frontera	Ejecutivo de ventas, 54 años, sin horas extra, ingreso 10.686	0,44	Yes	Se quedó: falsa alarma

Fuente: elaboración propia

Gráfico 3: Probabilidad devuelta por el servicio para cada empleado de prueba



Fuente: elaboración propia en Python

El tercer caso es el más ilustrativo. Está apenas por encima del corte de 0,40, de modo que el servicio lo marca aunque el empleado no haya renunciado. Con el corte estándar de 0,50 no aparecería. Esas falsas alarmas son el precio aceptado conscientemente en la Entrega 2 a cambio de detectar 8 de cada 10 renuncias reales: una conversación de retención de más cuesta mucho menos que perder a un empleado sin haberlo anticipado.

5. VALIDACIÓN DE LAS ENTRADAS

Un servicio que recibe datos de otros sistemas tiene que defenderse de datos mal formados. La validación se declara junto con el formato de entrada (Pydantic) y se aplica antes de que el modelo intervenga: si falta un campo, si un número llega como texto o si una variable categórica trae un valor inexistente, el servicio responde con el código 422 y el detalle exacto del problema.

Como prueba, se envió el empleado estable con el campo de horas extra en "quizás", un valor que no existe en el dataset. El servicio respondió con el código 422, el nombre del campo afectado y la indicación de que solo admite los valores No o Yes. Sin esta barrera, la codificación de variables dejaría pasar el valor desconocido como si fueran todas las categorías en cero y el modelo devolvería una probabilidad silenciosamente incorrecta: el error es preferible a una respuesta equivocada.

6. EJECUCIÓN Y DEMOSTRACIÓN

El servicio se levanta con un comando desde la carpeta del proyecto: `uvicorn api.app:app --reload`. Queda disponible en <http://127.0.0.1:8000> y la demostración se hace en <http://127.0.0.1:8000/docs>,

la pantalla del Gráfico 1, donde se puede completar el formulario de un empleado y ver la respuesta sin escribir código.

Las otras dos formas de consultarlo son el script `api/probar_ejemplos.py`, que envía los tres empleados de prueba, y una llamada directa desde la línea de comandos con `curl`, enviando el archivo `api/ejemplo.json`.

Para quien no quiera instalar nada, el cuaderno `notebooks/03_api_demo.ipynb` hace todo lo anterior dentro de Google Colab: levanta el servicio en segundo plano, comprueba que está vivo, envía los tres empleados, muestra el rechazo de un dato inválido y lo apaga.

7. CONCLUSIONES

El modelo entrenado en la Entrega 2 quedó disponible como un servicio consultable, con un formato de entrada y de salida definidos y validados, probado con empleados reales y documentado de forma interactiva. La Tabla 4 resume el cumplimiento de la consigna.

Tabla 4: Cumplimiento de la consigna de la Entrega 3

Punto	Qué se hizo
API con endpoint de predicción	FastAPI con POST <code>/predict</code> , más GET <code>/health</code> para control
Formato de entrada y salida definido	29 campos validados campo por campo; salida con la decisión y la probabilidad
Mismo pipeline que en entrenamiento	Se carga el artefacto de la Entrega 2, que contiene preparación, modelo y punto de corte
Probado con ejemplos	Tres empleados reales del grupo de examen: riesgo alto, estable y de frontera
Documentado	Documentación interactiva automática en <code>/docs</code> y cuaderno reproducible en Colab

Fuente: elaboración propia

Lo que faltaría para un uso real, siguiendo los temas de las últimas clases del curso: registrar cada versión del modelo con sus métricas para poder compararlas y volver atrás; vigilar que los datos que llegan al servicio no se alejen de los de entrenamiento; programar un reentrenamiento periódico; y un tablero de monitoreo con semáforos de calidad. A eso se suma la auditoría del sesgo por género y estado civil señalada en la Entrega 2, que en un despliegue real sería condición previa a poner el modelo a decidir sobre personas.

Con esta entrega se completa el recorrido del curso: entendimiento del problema, preparación de los datos, modelado y operacionalización.

8. LINK COLAB

El cuaderno de esta entrega, que levanta el servicio y lo consulta con los tres ejemplos, se ejecuta en Google Colab desde el siguiente enlace:

https://colab.research.google.com/github/gioaqp/attrition-uba/blob/main/notebooks/03_api_demo.ipynb

9. REFERENCIAS

- IBM HR Analytics Employee Attrition & Performance [dataset]. Kaggle.
<https://www.kaggle.com/datasets/pavansubhasht/ibm-hr-analytics-attrition-dataset>
- Barreto Marcano, A. (2026). Material de las clases 10 a 13 del curso M71V/M72V 02, UBA: consumo del modelo, documentación y monitoreo.
- FastAPI. Documentación oficial. <https://fastapi.tiangolo.com/>